

Lección 2: Test-Driven Development (TDD)

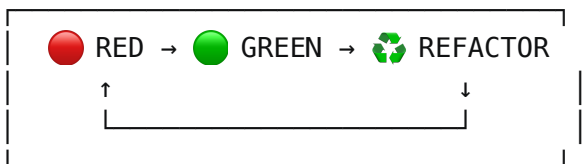
TDD con IA

Agenda

- Fundamentos de TDD
 - TDD + IA: Combinación Poderosa
 - Patrones de TDD
 - Arquitectura Emergente
 - Refactoring Seguro
-

¿Qué es TDD? (1/2)

Escribir tests ANTES que código de producción



¿Qué es TDD? (2/2)

Ciclo TDD:

-  **RED**: Escribe test que falle

-  **GREEN**: Código mínimo para pasar
-  **REFACTOR**: Mejora sin romper tests

No es solo testing, es diseño de software

Las Tres Leyes de TDD

1. **No escribas código de producción** hasta tener un test que falle
2. **No escribas más test** del necesario para fallar
3. **No escribas más código** del necesario para pasar el test

Disciplina, no solo técnica

TDD + IA: ¿Por qué?

La IA acelera el proceso TDD:

-  Genera tests comprehensivos
 -  Convierte requisitos en tests ejecutables
 -  Velocidad inicial aumentada
 -  Piensa en casos edge que olvidarías
-

Workflow TDD + IA

```
// 1. 🤖 IA genera especificación
describe('Shopping Cart', () => {
  it('should add new item to empty cart')
  it('should accumulate quantity for existing items')
  it('should throw error for invalid quantities')
})

// 2. 🛑 Implementar tests que fallen
// 3. 🟢 Código mínimo para pasar
// 4. ♻️ Refactorizar
```

Patrón: Fake It 'Til You Make It (1/2)

Estrategia TDD: Implementación más simple posible primero

```
// 🛑 Paso 1: Test falla
it('should calculate bulk discount', () => {
  expect(calculateBulkDiscount(item, 5)).toBe(15.0)
})
// 🟢 Paso 2: Implementación FAKE (hardcoded)
function calculateBulkDiscount() {
  return 15.0 // Sí, hardcoded es válido aquí!
}
```

Patrón: Fake It 'Til You Make It (2/2)

```
// 🛑 Paso 3: Nuevo test obliga a generalizar
it('should calculate bulk discount for 10 items', () => {
  expect(calculateBulkDiscount(item, 10)).toBe(30.0)
})
// 🟢 Paso 4: Ahora SÍ necesitamos lógica real
function calculateBulkDiscount(item, quantity) {
  if (quantity >= 5) return item.price * quantity * 0.1
  return 0
}
```

Lección: Más tests → Fuerzan implementación real

Patrón: Triangulation (1/2)

Usar múltiples tests para triangular hacia la solución correcta

```
// Test 1: Caso límite inferior
it('returns 0 for < 5 items', () => {
  expect(calculateBulkDiscount(item, 3)).toBe(0)
})
// Test 2: Caso límite exacto
it('calculates discount for exactly 5 items', () => {
  expect(calculateBulkDiscount(item, 5)).toBe(15.0)
})
```

```
})
```

Patrón: Triangulation (2/2)

```
// Test 3: Caso general (confirma lógica)
it('calculates discount for 10 items', () => {
  expect(calculateBulkDiscount(item, 10)).toBe(30.0)
})
// Ahora la implementación DEBE ser correcta:
function calculateBulkDiscount(item, quantity) {
  return quantity >= 5 ? item.price * quantity * 0.1 : 0
}
```

3 tests triangularon la solución correcta

Arquitectura Emergente

TDD diseña la arquitectura:

```
// Los tests definen la API antes de que exista
describe('Cart Operations', () => {
  it('should add items immutably', () => {
    const updatedCart = addItemToCart(originalCart, item, 1)
    expect(originalCart).not.toBe(updatedCart)
  })
})
```

El test dicta:

- Signature de la función
 - Comportamiento (inmutabilidad)
 - Contratos
-

Funciones Puras Emergentes (1/2)

Función Pura (pure function): Siempre retorna mismo resultado para mismos inputs, sin efectos secundarios (side effects)

```
// ❌ Difícil de testear (impura)
function addToCart(item, quantity) {
  globalCart.push(item) // Muta estado global
  updateLocalStorage() // Efecto secundario (side effect)
}
```

Funciones Puras Emergentes (2/2)

```
// ✅ Fácil de testear (TDD lleva a esto)
function addItemToCart(cart, item, quantity) {
  return {
    items: [...cart.items, { ...item, quantity }],
  } // Función pura: sin mutación, sin side effects
}
```

Ejercicio 1: RED - Test que Falla

Prompt:

Actúa como un desarrollador practicando TDD estricto.

CONTEXT0: En TDD, escribes el **test** ANTES que el código de producción.
Primera ley de TDD: "No escribas código de producción hasta tener un **test que falle**". Este es el paso RED del ciclo Red-Green-Refactor.

TAREA: Crea **test** para calculateTax que DEBE FALLAR.

TEST SPECIFICATIONS:

- Función a probar: calculateTax(amount: number, taxRate: number): number
- Test **case**: calculateTax(100, 10) debe retornar 10
- Framework: Vitest (describe, it, expect)
- Estructura: AAA pattern con comentarios

IMPORTANTE – PASO RED:

- NO implementes la función calculateTax aún
- El **test** DEBE fallar con "ReferenceError: calculateTax is not defined"

- Esto prueba que seguiste TDD correctamente

ARCHIVOS:

- src/shared/utils/calculateTax.test.ts (solo el `test`)

VALIDACIÓN: ejecuta `pnpm test` → debe fallar con error `"not defined"`

Aprende: TDD RED - test primero, código después

Ejercicio 2: GREEN - Fake It

Prompt:

Actúa como un desarrollador usando el patrón `"Fake It 'Til You Make It"`.

CONTEXTO: En el paso GREEN de TDD, escribes el código MÍNIMO necesario para hacer pasar el `test`. `"Fake It"` significa que puedes usar valores hardcoded (constantes fijas). No necesitas lógica real todavía. Tercera ley de TDD: `"No escribas más código del necesario para pasar el test"`.

TAREA: Implementa `calculateTax` usando Fake It pattern.

IMPLEMENTACIÓN REQUIREMENTS:

- Función: `calculateTax(amount: number, taxRate: number): number`
- Implementación: `return 10` (sí, hardcoded es VÁLIDO aquí)
- Objetivo: hacer pasar el `test` del Ejercicio 1
- NO implementes lógica real (`amount * taxRate / 100`) todavía

IMPORTANTE - PASO GREEN:

- El código `"malo"` es intencional en este paso
- Hardcoded es válido en TDD cuando hace pasar el `test`
- El siguiente `test` (Triangulation) forzará lógica real

ARCHIVOS:

- src/shared/utils/calculateTax.ts (implementación fake)

VALIDACIÓN: ejecuta `pnpm test` → `test` debe PASAR 

Aprende: Fake It - implementación mínima,

hardcoded válido en TDD

Ejercicio 3: Triangulation

Prompt:

Actúa como un desarrollador usando el patrón Triangulation.

CONTEXT0: Triangulation es usar múltiples tests para forzar la implementación correcta. Un solo `test` permite "Fake It" (hardcoded). Dos o más tests diferentes obligan a generalizar la lógica. Los tests "triangular" hacia la solución real.

TAREA: Agrega segundo `test` y refactoriza hacia implementación real.

TEST SPECIFICATIONS:

- Nuevo `test case`: `calculateTax(200, 15)` debe retornar 30
- Mantén el `test` anterior: `calculateTax(100, 10)` debe retornar 10
- Ambos tests deben estar en el mismo describe block

CICLO TDD:

1.  RED: Agregar segundo `test` → FALLA (hardcoded retorna 10, no 30)
2.  GREEN: Refactorizar implementación a lógica real
3. Fórmula correcta: $\text{amount} * (\text{taxRate} / 100)$
4.  Ambos tests deben PASAR

IMPORTANTE – TRIANGULATION:

- El hardcoded YA NO funciona con 2 tests diferentes
- Los tests fuerzan la implementación correcta
- Esto es más confiable que adivinar la lógica

ARCHIVOS:

- `src/shared/utils/calculateTax.test.ts` (agregar segundo `test`)
- `src/shared/utils/calculateTax.ts` (refactorizar a lógica real)

VALIDACIÓN: ejecuta `pnpm test` → ambos tests deben PASAR 

Aprende: Triangulation - múltiples tests fuerzan lógica correcta

Refactoring Seguro con TDD

Green Bar Pattern (patrón barra verde): Mantener siempre la barra verde (todos los tests pasando)

```
//  Durante refactoring, tests siempre pasan
describe('Before Refactor', () => {
  it('calculates discount', () => {
    /*  */
  })
})

// Refactor paso a paso
function calculate(item, quantity) {
  // Paso 1: Extraer constante 
  // Paso 2: Añadir validación 
  // Paso 3: Mejorar precisión 
}
```

Métricas de TDD

Cuantitativas (medibles):

- Test First: 95%+ código después de test
- Coverage: 95%+ es típico en TDD
- Defect Rate (tasa de defectos): 40-80% menos bugs

Cualitativas (perceptibles):

-  Mejor diseño emergente
-  Documentación viviente
-  Refactoring sin miedo
-  Retroalimentación rápida

Puntos Clave

1. **Test First:** Siempre test antes que código
2. **Red-Green-Refactor:** El ciclo sagrado de TDD
3. **Asistencia IA:** Acelera generación de tests
4. **Diseño:** TDD mejora arquitectura emergente
5. **Funciones Puras:** TDD favorece funciones puras
6. **Documentación Viviente:** Tests como especificación ejecutable

